

UNIVERSIDADE FEDERAL DE SERGIPE
CENTRO DE CIÊNCIAS EXATAS E TECNOLOGIA
DEPARTAMENTO DE COMPUTAÇÃO

GABRIEL RODRIGUES DOS SANTOS

SISTEMA DO DENATRAN COM MICROSSERVIÇOS E MQTT: atividade 01 de publicar-assinar

SÃO CRISTÓVÃO
2026

GABRIEL RODRIGUES DOS SANTOS

SISTEMA DO DENATRAN COM MICROSSERVIÇOS E MQTT: atividade 01 de publicar-assinar

Relatório da Atividade 01 (Pub/Sub) apresentado à disciplina Sistemas Distribuídos (COMP0470) do Departamento de Computação da Universidade Federal de Sergipe.

Professor: Prof. Dr. Rafael Oliveira Vasconcelos.

SÃO CRISTÓVÃO
2026

SUMÁRIO

1 INTRODUÇÃO **3**

2 ARQUITETURA DA SOLUÇÃO **4**

3 COMUNICAÇÃO PELO MQTT **5**

3.1 Requisição e resposta **5**

3.2 Eventos entre os serviços **6**

3.3 Confiabilidade e desacoplamento temporal **6**

4 REGRAS ADOTADAS **8**

5 IMPLEMENTAÇÃO **9**

6 EXECUÇÃO COM DOCKER **10**

7 TESTES **11**

7.1 Teste automático **11**

7.2 Serviço fora do ar **11**

8 CONSIDERAÇÕES FINAIS **13**

REFERÊNCIAS **14**

1 INTRODUÇÃO

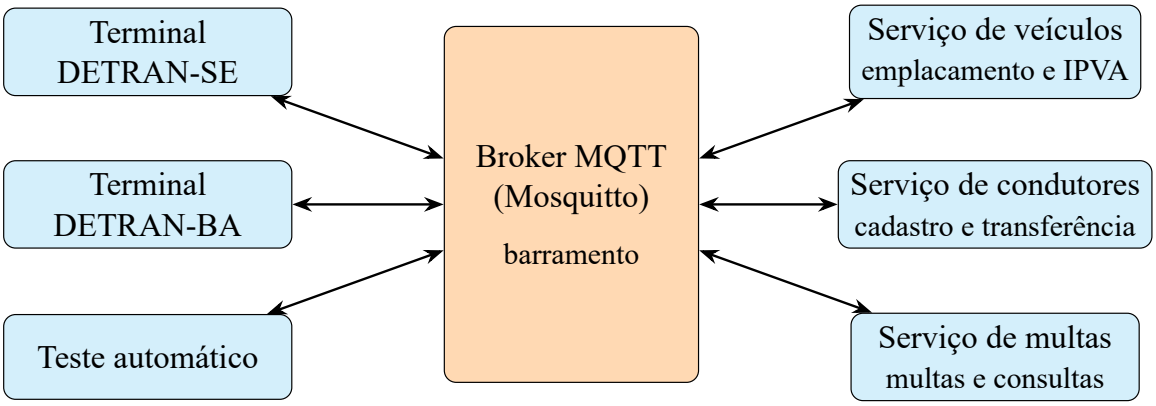
O Departamento Nacional de Trânsito (DENATRAN) precisa de um sistema para gerenciar os veículos e condutores de todos os estados, com as funcionalidades de emplacar veículo, calcular o Imposto sobre a Propriedade de Veículos Automotores (IPVA) com alíquota de 2%, transferir o proprietário, cadastrar condutor, lançar multa, listar os veículos emplacados em um ano, listar as multas de um veículo (com os dados do condutor que levou cada multa), as multas de um condutor em um ano, as multas lançadas em um ano e os 5 condutores com as maiores pontuações. O sistema deve ser implementado no estilo de microsserviços, usando o protocolo *Message Queuing Telemetry Transport* (MQTT) como barramento de comunicação.

Este relatório descreve a arquitetura adotada, a comunicação entre os serviços, as regras de negócio, a implementação, a execução com Docker e os testes realizados.

2 ARQUITETURA DA SOLUÇÃO

O sistema foi dividido em três microsserviços, seguindo o agrupamento sugerido no enunciado: emplacamento e IPVA, cadastro e transferência, e multas. Todos se conectam a um broker MQTT (Mosquitto), que é o único ponto de contato entre eles. Os departamentos estaduais de trânsito (DETRANs) usam o sistema por um terminal que também se conecta ao broker. A Figura 1 mostra a organização e o Quadro 1, as funcionalidades de cada serviço.

Figura 1 – Organização do sistema



Fonte: Elaborada pelo autor (2026).

Quadro 1 – Microsserviços e funcionalidades

Microsserviço	**Funcionalidades**
`veiculos`	emplacar veículo; calcular IPVA; informar os veículos emplacados em um ano
`condutores`	cadastrar condutor; transferir proprietário de um veículo
`multas`	lançar multa; multas de um veículo; multas de um condutor em um ano; multas lançadas em um ano; 5 condutores com mais pontos

Fonte: Elaborado pelo autor (2026).

Cada microsserviço guarda apenas os seus próprios dados, em um arquivo *JavaScript Object Notation* (JSON) que fica em um volume do Docker, seguindo a ideia de gerenciamento descentralizado de dados dos microsserviços (Lewis; Fowler, 2014). Nenhum serviço acessa os dados de outro diretamente: tudo o que um serviço precisa saber sobre os outros chega por mensagens publicadas no broker.

3 COMUNICAÇÃO PELO MQTT

O MQTT segue o modelo publicar-assinar baseado em tópicos (Banks; Gupta, 2014): quem publica envia a mensagem para um tópico e o broker entrega a mensagem a todos que assinaram aquele tópico, como no modelo publicar-assinar visto em aula (Coulouris *et al.*, 2013). Os tópicos foram organizados de forma hierárquica, no formato `denatran/<serviço>/<operação>`. Com isso cada serviço assina todas as suas operações com um único filtro usando o curinga `+`, que corresponde a um nível do tópico, e o serviço de multas assina todos os eventos com o curinga `#`, que corresponde a todos os níveis abaixo de `denatran/eventos` (HiveMQ, 2019). As mensagens são em JSON. O Quadro 2 lista os tópicos.

Quadro 2 – Tópicos usados no barramento

Tópico	**Campos da mensagem**
`denatran/veiculos/emplacar`	placa, modelo, valor, cpf
`denatran/veiculos/ipva`	placa
`denatran/veiculos/emplacados-no-ano`	ano
`denatran/condutores/cadastrar`	cpf, nome
`denatran/condutores/transferir`	placa, cpf do novo dono
`denatran/multas/lancar`	ano, descricao, pontuacao, placa
`denatran/multas/por-veiculo`	placa, ano (opcional)
`denatran/multas/por-condutor`	cpf, ano
`denatran/multas/por-ano`	ano
`denatran/multas/ranking`	sem campos
`denatran/eventos/condutor-cadastrado`	cpf, nome
`denatran/eventos/veiculo-emplacado`	placa, modelo, cpf
`denatran/eventos/veiculo-transferido`	placa, cpfAnterior, cpfNovo
`denatran/respostas/<id do cliente>`	idRequisicao, sucesso, mensagem, dados

Fonte: Elaborado pelo autor (2026).

3.1 Requisição e resposta

Na versão 3.1.1 do MQTT, usada no trabalho, não existe o conceito de resposta, então a resposta também é uma publicação. Cada terminal gera um identificador próprio e assina o tópico `denatran/respostas/<id>`. Ao fazer uma requisição, o terminal publica no tópico da operação uma mensagem com os dados, um `idRequisicao` e o campo `responderPara` com o seu tópico de resposta. O serviço que assina aquele tópico processa a requisição e publica a resposta no tópico indicado. O terminal espera até 10 segundos pela resposta com o mesmo `idRequisicao`. O terminal não sabe qual serviço vai atender nem onde ele está, conhece apenas o tópico, o que caracteriza o desacoplamento espacial.

3.2 Eventos entre os serviços

Algumas funcionalidades dependem de dados de outro serviço. Para emplacar, o serviço de veículos precisa saber se o dono, identificado pelo Cadastro de Pessoas Físicas (CPF), já foi cadastrado; para transferir, o serviço de condutores precisa saber quem é o dono atual da placa; para lançar uma multa, o serviço de multas precisa saber o dono do veículo e o nome do condutor. Em vez de um serviço consultar o outro, cada serviço publica um evento quando os seus dados mudam, e os serviços interessados assinam esses eventos e mantêm uma cópia local só do que precisam (Quadro 3). Assim um evento publicado uma vez pode ser entregue a vários assinantes (comunicação de um para muitos), e quem publica não precisa saber quem usa a informação.

Quadro 3 – Eventos publicados e serviços que os assinam

Evento	**Publicado por**	**Assinado por**
`condutor-cadastrado`	condutores	veículos (CPFs válidos) e multas (nome do condutor)
`veiculo-emplacado`	veículos	condutores (dono da placa) e multas (dono da placa)
`veiculo-transferido`	condutores	veículos (atualiza o proprietário) e multas (novo dono)

Fonte: Elaborado pelo autor (2026).

A Listagem 1 foi capturada com o `mosquitto_sub` assinando `denatran/#` durante o cadastro de um condutor. Aparecem a requisição do terminal, o evento publicado pelo serviço de condutores e a resposta enviada ao terminal.

Listagem 1 – Mensagens no barramento durante o cadastro de um condutor

```
denatran/condutores/cadastrar {"cpf":"08231546710","nome":"Mariana Costa",
  "idRequisicao":"8bf865d0-...",
  "responderPara":"denatran/respostas/detran-se-8054b60f","detran":"SE"}
denatran/eventos/condutor-cadastrado {"cpf":"08231546710","nome":"Mariana Costa"}
denatran/respostas/detran-se-8054b60f {"idRequisicao":"8bf865d0-...","sucesso":true,
  "mensagem":"condutor Mariana Costa cadastrado",
  "dados":[{"cpf":"08231546710","nome":"Mariana Costa"}]}
```

Fonte: Elaborada pelo autor (2026).

3.3 Confiabilidade e desacoplamento temporal

Todas as mensagens são publicadas com nível 1 de qualidade de serviço (QoS), em que o broker confirma o recebimento e a entrega é feita pelo menos uma vez (Banks; Gupta, 2014; HiveMQ, 2015a). Os microsserviços se conectam com identificador fixo e sessão persistente, com `cleanSession = false` (HiveMQ, 2015b). Dessa forma o broker mantém as assinaturas de cada serviço e guarda as mensagens que chegam enquanto ele está desconectado, entregando

tudo quando ele volta. Os serviços não precisam estar em execução ao mesmo tempo para trocar eventos, e uma requisição feita a um serviço parado é processada quando ele é reiniciado. O Mosquitto foi configurado com persistência em disco para não perder essas mensagens se o próprio broker reiniciar (Eclipse Foundation, 2026a).

4 REGRAS ADOTADAS

Na implementação, foram adotadas as seguintes regras:

a) o ano de emplacamento é o ano em que o veículo foi emplacado no sistema;

b) só é possível emplacar um veículo para um CPF já cadastrado, e a mesma placa não pode ser emplacada duas vezes;

c) são aceitas placas no formato antigo (ABC1234) e no Mercosul (ABC1D23), e o CPF deve ter 11 dígitos, desconsiderando pontos e traço;

d) o IPVA é 2% do valor do veículo;

e) a multa é atribuída ao proprietário do veículo no momento do lançamento, então, depois de uma transferência, as multas antigas continuam com o dono anterior e as novas vão para o novo dono, e as consultas mostram os dados do condutor que levou cada multa;

f) na consulta de multas de um veículo o ano é opcional e, se não for informado, são listadas todas as multas do veículo;

g) o ranking soma os pontos de todas as multas de cada condutor e mostra os 5 maiores em ordem decrescente.

5 IMPLEMENTAÇÃO

O sistema foi escrito em Java 17, com a biblioteca Eclipse Paho para o MQTT (Eclipse Foundation, 2026b) e o Gson para o JSON (Google, 2026). As classes principais são:

a) `Microsservico`: classe base dos serviços, que conecta ao broker com sessão persistente, assina os tópicos, trata as mensagens uma de cada vez em uma única thread, envia as respostas e grava os dados do serviço em arquivo;

b) `ServicoVeiculos`, `ServicoCondutores` e `ServicoMultas`: os três microsserviços, cada um com as suas operações e o tratamento dos eventos que assina;

c) `Cliente`: envia as requisições e aguarda as respostas no tópico do cliente;

d) `Terminal`: menu no console com as 10 funcionalidades;

e) `Teste`: teste automático de todas as funcionalidades.

O serviço de multas, por exemplo, é iniciado assinando as suas operações e todos os eventos (Listagem 2), e o lançamento de uma multa usa a cópia local do proprietário de cada placa, mantida pelos eventos (Listagem 3).

Listagem 2 – Início do serviço de multas

```
new ServicoMultas().iniciar(
        Mqtt.MULTAS + "+",      // denatran/multas/+
        Mqtt.EVENTOS + "#");    // denatran/eventos/#
```

Fonte: Elaborada pelo autor (2026).

Listagem 3 – Lançamento de uma multa

```
String cpf = dados.proprietarios.get(placa);
if (cpf == null) {
    throw new IllegalArgumentException("veiculo " + placa + " nao encontrado");
}

// a multa fica com o dono do veículo no momento do lançamento
Multa multa = new Multa(ano, descricao, pontuacao, placa, cpf);
dados.multas.add(multa);
salvar(dados);
```

Fonte: Elaborada pelo autor (2026).

6 EXECUÇÃO COM DOCKER

O `Dockerfile` compila o projeto com o Maven e gera uma imagem com o Java e o `.jar`, usada por todos os programas (Docker, 2026c, 2026d). O `docker-compose.yml` define o broker `mosquitto`, os serviços `veiculos`, `condutores` e `multas`, com um volume de dados para cada um, e os clientes `terminal` e `teste`, que são executados sob demanda. Os microsserviços só iniciam depois que o broker aceita conexões (Docker, 2026a, 2026b). A Listagem 4 mostra os comandos de execução.

Listagem 4 – Comandos de execução

```
docker compose up -d --build        # sobe o broker e os tres microsservicos
docker compose run --rm terminal    # abre o menu do DETRAN-SE
docker compose run --rm teste       # teste automatico (servicos sem dados)
docker compose logs -f              # log dos servicos
docker compose down                 # encerra (com -v apaga os dados)
```

Fonte: Elaborada pelo autor (2026).

7 TESTES

7.1 Teste automático

O programa `Teste` cadastra seis condutores, emplaca seis veículos, lança nove multas em 2024, 2025 e 2026, transfere um veículo e faz todas as consultas, conferindo cada resposta. Também confere as situações de erro: CPF repetido, placa repetida, emplacamento para CPF não cadastrado, multa para veículo inexistente e transferência para quem já é dono. As 40 verificações passaram. A Listagem 5 mostra um trecho da saída.

Listagem 5 – Trecho da saída do teste automático

```
denatran/veiculos/ipva {"placa":"ABC1D23"}
OK: IPVA do veiculo ABC1D23: R$ 900,00
  placa: ABC1D23 | modelo: Gol | valor: R$ 45.000,00 | aliquota: 2% | ipva: R$ 900,00
  [OK] IPVA de R$ 45.000,00 = R$ 900,00
denatran/condutores/transferir {"placa":"ABC1D23","cpf":"32678904152"}
OK: veiculo ABC1D23 transferido de Mariana Costa para Ricardo Menezes
  [OK] transferir ABC1D23
denatran/multas/por-veiculo {"placa":"ABC1D23","ano":""}
OK: 3 multa(s) do veiculo ABC1D23
  ano: 2024 | descricao: Estacionar em local proibido | pontos: 4 | ... |
    condutor: Mariana Costa
  ano: 2025 | descricao: Excesso de velocidade ate 20% | pontos: 4 | ... |
    condutor: Mariana Costa
  ano: 2026 | descricao: Excesso de velocidade ate 20% | pontos: 4 | ... |
    condutor: Ricardo Menezes
  [OK] multas antigas com Mariana e a nova com Ricardo
denatran/multas/ranking {}
OK: condutores com mais pontos em multas (ate 5)
  posicao: 1 | condutor: Joao Pedro Almeida | cpf: 70455218903 | pontos: 14
  posicao: 2 | condutor: Ricardo Menezes | cpf: 32678904152 | pontos: 9
  posicao: 3 | condutor: Mariana Costa | cpf: 08231546710 | pontos: 8
  posicao: 4 | condutor: Tatiane Farias | cpf: 51920837461 | pontos: 7
  posicao: 5 | condutor: Luana Batista | cpf: 96014372285 | pontos: 5

Todas as verificacoes passaram
```

Fonte: Elaborada pelo autor (2026).

7.2 Serviço fora do ar

Para testar o desacoplamento temporal, o serviço de multas foi parado e, pelo terminal, foram cadastrados um condutor e um veículo. Os eventos `condutor-cadastrado` e `veiculo-emplacado` foram publicados com o serviço de multas desligado. Depois o serviço foi iniciado de novo e uma multa foi lançada para o veículo, com os comandos e o resultado da Listagem 6.

Listagem 6 – Teste com o serviço de multas parado

```
docker compose stop multas
docker compose run --rm terminal     # opcao 4 (condutor) e opcao 1 (emplacar SEA2B34)
docker compose start multas
docker compose run --rm terminal     # opcao 5 (lancar multa para SEA2B34)

OK: multa lancada para o veiculo SEA2B34 (condutor Fernanda Rocha)
```

Fonte: Elaborada pelo autor (2026).

A multa foi registrada para o condutor correto, o que mostra que o broker guardou os eventos e os entregou ao serviço de multas quando ele voltou. Também foi feito o teste de reiniciar os três serviços: os dados continuaram disponíveis, porque cada serviço carrega o seu arquivo ao iniciar.

8 CONSIDERAÇÕES FINAIS

As dez funcionalidades pedidas foram implementadas em três microsserviços que se comunicam apenas pelo broker MQTT. Cada serviço guarda os seus dados e recebe o que precisa dos outros por eventos, e com QoS 1 e sessão persistente um serviço pode ficar parado sem perder as mensagens destinadas a ele. O código, o `Dockerfile`, o `docker-compose.yml` e os comandos de execução estão no projeto entregue, com instruções no arquivo `README.md`.

REFERÊNCIAS

BANKS, Andrew; GUPTA, Rahul (ed.). **MQTT Version 3.1.1**. [*S. l.*]: OASIS, 2014. OASIS Standard. Disponível em: https://docs.oasis-open.org/mqtt/mqtt/v3.1.1/os/mqtt-v3.1.1-os.html. Acesso em: 29 set. 2026.

COULOURIS, George; DOLLIMORE, Jean; KINDBERG, Tim; BLAIR, Gordon. **Sistemas distribuídos**: conceitos e projeto. 5. ed. Porto Alegre: Bookman, 2013.

DOCKER. **Compose file reference**. [*S. l.*], 2026a. Disponível em: https://docs.docker.com/reference/compose-file/. Acesso em: 29 set. 2026.

DOCKER. **Control startup and shutdown order in Compose**. [*S. l.*], 2026b. Disponível em: https://docs.docker.com/compose/how-tos/startup-order/. Acesso em: 29 set. 2026.

DOCKER. **Dockerfile reference**. [*S. l.*], 2026c. Disponível em: https://docs.docker.com/reference/dockerfile/. Acesso em: 29 set. 2026.

DOCKER. **Multi-stage builds**. [*S. l.*], 2026d. Disponível em: https://docs.docker.com/build/building/multi-stage/. Acesso em: 29 set. 2026.

ECLIPSE FOUNDATION. **Eclipse Mosquitto**: an open source MQTT broker. [*S. l.*]: GitHub, 2026a. Disponível em: https://github.com/eclipse-mosquitto/mosquitto. Acesso em: 29 set. 2026.

ECLIPSE FOUNDATION. **Eclipse Paho Java MQTT client library**. [*S. l.*]: GitHub, 2026b. Disponível em: https://github.com/eclipse-paho/paho.mqtt.java. Acesso em: 29 set. 2026.

GOOGLE. **Gson**: a Java serialization/deserialization library to convert Java Objects into JSON and back. [*S. l.*]: GitHub, 2026. Disponível em: https://github.com/google/gson. Acesso em: 29 set. 2026.

HIVEMQ. **What is MQTT quality of service (QoS) 0, 1, & 2?** [*S. l.*], 2015a. (MQTT Essentials, part 6). Disponível em: https://www.hivemq.com/blog/mqtt-essentials-part-6-mqtt-quality-of-service-levels/. Acesso em: 29 set. 2026.

HIVEMQ. **Understanding persistent sessions and clean sessions**. [*S. l.*], 2015b. (MQTT Essentials, part 7). Disponível em: https://www.hivemq.com/blog/mqtt-essentials-part-7-persistent-session-queuing-messages/. Acesso em: 29 set. 2026.

HIVEMQ. **MQTT topics, wildcards, & best practices**. [*S. l.*], 2019. (MQTT Essentials, part 5). Disponível em: https://www.hivemq.com/blog/mqtt-essentials-part-5-mqtt-topics-best-practices/. Acesso em: 29 set. 2026.

LEWIS, James; FOWLER, Martin. **Microservices**: a definition of this new architectural term. [*S. l.*], 25 mar. 2014. Disponível em: https://martinfowler.com/articles/microservices.html. Acesso em: 29 set. 2026.